Sprint 8 : Objects, Methods, JSON, and Safe Property Access.

What Sprint 8 Is About

Concept Explanation (Sinhala): Sprint 8 එකෙන් අපි ප්‍රධාන වශයෙන්ම ඉගෙන ගන්නේ **structured data** (ක්‍රමවත්ව සකස් කරපු දත්ත) එක්ක වැඩ කරන්නේ කොහොමද කියන එකයි.

මීට කලින් sprints වලදී ඔයා තනි තනි values, variables, strings, numbers, booleans, conditions, functions, arrays සහ loops වගේ දේවල් ඉගෙන ගත්තා. හැබැයි ඇත්තම ලෝකයේ භාවිත වන application එකක data තනි තනිව (single values විදිහට) තියාගන්නේ නැහැ. ඒවා එකිනෙකට සම්බන්ධ කරලා එකම structure එකක් විදිහට තබා ගන්නවා.

Code Block:

```
// මේක කලින් ක්‍රමය (තනි තනි values):
const name = "Alice";
const age = 25;
const isActive = true;

// මේක Sprint 8 ක්‍රමය (Objects භාවිතයෙන් සම්බන්ධ කරපු structure එකක්):
const user = {
  name: "Alice",
  age: 25,
  isActive: true
};
```

Code Breakdown (Sinhala):

- කලින් ක්‍රමයේදී `name`, `age` සහ `isActive` කියන දත්ත වෙන වෙනම variables විදිහට තිබුණේ. ඒවා අතර පැහැදිලි සම්බන්ධයක් code එක ඇතුළේ පෙනෙන්න නැහැ.
- දෙවැනි ක්‍රමයේදී `user` කියන එකම **Object** එකක් ඇතුළට මේ දත්ත සියල්ලම එකතු කරලා තියෙනවා. මේකෙන් අපිට දත්ත පහසුවෙන් පාලනය කරන්න සහ ආරක්ෂිතව access කරන්න පුළුවන් වෙනවා.

Part B: The Core Idea of Objects

1. What Is an Object?

Concept Explanation (Sinhala): Object එකක් කියන්නේ එකිනෙකට සම්බන්ධ දත්ත රැසක් එකට එකතු කරලා තියාගන්න පුළුවන් variable value එකක්. Object එකක් ඇතුළේ data store වෙන්නේ **key-value pairs** (යතුර සහ අගය යුගල) විදිහටයි.

Code Block:

```
const user = {
  name: "Alice",
  age: 25,
  active: true
};
```

Code Breakdown (Sinhala): මේ `user` කියන Object එක ඇතුළේ properties 3ක් තියෙනවා:

- `name` කියන්නේ **key** එකයි, `"Alice"` කියන්නේ එහි **value** එකයි.
- `age` කියන්නේ **key** එකයි, `25` කියන්නේ එහි **value** එකයි.
- `active` කියන්නේ **key** එකයි, `true` කියන්නේ එහි **value** එකයි.

පහසු අර්ථ දැක්වීමක් විදිහට, Object එකක් කියන්නේ එකිනෙකට සම්බන්ධ තොරතුරු එක තැනක බහාලන කන්ටේනරයක් (container) වගේ දෙයක්. (උදාහරණ: users, students, products, books, tasks, settings, profiles, game characters වගේ ඕනෑම දෙයක් නිරූපණය කරන්න මේවා පාවිච්චි කරන්න පුළුවන්).

2. Why Objects Are Useful

Concept Explanation (Sinhala): අපි Objects පාවිච්චි නොකළොත්, එකිනෙකට සම්බන්ධ වෙන්න ඕන දත්ත විසිරිලා ගිහින් අවුල් ජාලාවක් වෙන්න පුළුවන්.

Code Block:

```
// Less organized (අඩු සංවිධානාත්මක බවක් ඇත):
const studentName = "Sangeeth";
const studentAge = 16;
const studentCourse = "JavaScript";
const studentActive = true;

// Better organized (වඩාත් සංවිධානාත්මකයි):
const student = {
  name: "Sangeeth",
  age: 16,
  course: "JavaScript",
  active: true
};
```

Code Breakdown (Sinhala):

- පළමු ක්‍රමයේදී variables 4ක් තියෙනවා. හැබැයි මේවා එකම student කෙනෙකුගේ කියලා අඳුරගන්න අමාරුයි.
- දෙවැනි ක්‍රමයේදී `student` කියන එකම structure එකක් ඇතුළට සියල්ල ගොනු කරලා තියෙනවා. දැන් අපිට තනි structure එකක් විදිහට මේ දත්ත කළමනාකරණය කරන්න පුළුවන්.

3. Object Syntax & Properties

Concept Explanation (Sinhala): Object එකක් සාමාන්‍යයෙන් ලියන්නේ සඟල වරහන් `{}` (curly braces) ඇතුළේ.

Code Block:

```
const book = {
  title: "JavaScript Basics",
  pages: 200,
  finished: false
};
```

Code Breakdown (Sinhala):

- `const book` : `book` නමින් variable එකක් හදනවා.
- `{ ... }` : Object එකක් නිර්මාණය කරනවා.
- `title` : **property name** එකක්, නැත්නම් **key** එකක්.
- `"JavaScript Basics"` : `title` property එක ඇතුළේ store කරලා තියෙන **value** එක.
- `:` (colon) : Key එක සහ value එක වෙන් කරනවා.
- `,` (comma) : Properties එකිනෙක වෙන් කරනවා.

මතක තබාගන්න: Object එකක් ඇතුළේ තියෙන එකම එක key-value pair එකකට අපි **property** එකක් කියලා කියනවා.

4. Object Values Can Have Many Types

Concept Explanation (Sinhala): Object එකක් ඇතුළේ ඕනෑම data type එකක value එකක් store කරන්න පුළුවන්. (strings, numbers, booleans, null, undefined, arrays, තවත් nested objects හෝ functions වුවත් ගබඩා කළ හැක).

Code Block:

```
const example = {
  text: "hello",
  number: 42,
  boolean: true,
  empty: null,
  missing: undefined,
  list: ["HTML", "CSS", "JavaScript"],
  nestedObject: {
    level: "beginner"
  },
  sayHi: function() {
    return "Hi";
  }
};
```

Code Breakdown (Sinhala):

- `text` ගබඩා කරන්නේ string එකක්.
- `list` ගබඩා කරන්නේ array එකක්.
- `nestedObject` ගබඩා කරන්නේ තවත් object එකක්.
- `sayHi` කියන්නේ function එකක් (මේ වගේ object එකක් ඇතුළේ තියෙන function එකකට අපි **method** එකක් කියලත් කියනවා).

Part C: Reading Object Properties

Object එකක තියෙන properties කියවන්න (read කරන්න) ප්‍රධාන ක්‍රම 2ක් තියෙනවා:

1. **Dot notation** (තිත භාවිතයෙන්)
2. **Bracket notation** (වරහන් භාවිතයෙන්)

1. Dot Notation

Concept Explanation (Sinhala): අපි ගන්න යන property name එක සරල, කිසිම ඉඩක් (space) නැති, direct ලියන්න පුළුවන් එකක් නම් අපි **dot notation** පාවිච්චි කරනවා.

Code Block:

```
const student = {
  name: "Sangeeth",
  course: "JavaScript"
};

console.log(student.name);
console.log(student.course);
// Output:
// Sangeeth
// JavaScript
```

Code Breakdown (Sinhala): `student.name` කියන එකෙන් අදහස් වෙන්නේ "student කියන Object එක ඇතුළට ගිහින්, එහි තියෙන `name` property එකේ අගය අරන් දෙන්න" කියන එකයි.

2. Bracket Notation

Concept Explanation (Sinhala): අපිට **bracket notation** පාවිච්චි කරන්නම වෙන අවස්ථා කිහිපයක් තියෙනවා:

- Property name එක මැද හිස්තැන් (spaces) තියෙනවා නම්.
- Property name එකේ special characters තියෙනවා නම්.
- Property name එක වෙනත් variable එකක store කරලා තියෙනවා නම් **(dynamic property access)**.

Code Block:

```
const student = {
  name: "Sangeeth",
  "favorite language": "JavaScript"
};

console.log(student["favorite language"]); // Output: JavaScript
// console.log(student.favorite language); // මේක වැරදියි! SyntaxError එකක් එනවා.
```

Code Breakdown (Sinhala): `favorite language` කියන key එක මැද space එකක් තියෙන නිසා අපිට ඒක තිතක් පාවිච්චි කරලා ලියන්න බැහැ. ඒ නිසා අපි ඒක square brackets `[]` ඇතුළේ string එකක් විදිහට දෙනවා: `student["favorite language"]`.

3. Dynamic Property Access

Concept Explanation (Sinhala): Dynamic property access කියන්නේ අපිට ඕන කරන property key එක කෙළින්ම ලියන්නේ නැතිව, වෙනත් variable එකක් හරහා ලබා දීමයි. මෙතනදී අනිවාර්යයෙන්ම **bracket notation** පාවිච්චි කරන්න ඕනේ.

Code Block:

```
const student = {
  name: "Sangeeth",
  age: 16,
  course: "JavaScript"
};

const key = "course";
console.log(student[key]); // Output: JavaScript
```

Code Breakdown (Sinhala): `student[key]` කියන කොටසේදී, JavaScript මුලින්ම `key` කියන variable එක ඇතුළේ තියෙන අගය බලනවා (ඒක තමයි `"course"`). ඊට පස්සේ ඒක `student["course"]` බවට පත් කරලා, අදාළ value එක වෙන `"JavaScript"` ලබා දෙනවා.

4. Dot Notation vs Bracket Notation (වැදගත්ම වෙනස)

Concept Explanation (Sinhala): Dot notation සහ Bracket notation පාවිච්චි කරනකොට වෙන වැරදීමක් පහතින් බලමු.

Code Block:

```
const student = {
  name: "Sangeeth",
  course: "JavaScript"
};

const key = "course";

console.log(student.key);  // Output: undefined
console.log(student[key]); // Output: JavaScript
```

Code Breakdown (Sinhala):

- `student.key` කියපු ගමන් JavaScript කෙළින්ම හොයන්නේ `"key"` කියලා අකුරු හතරකින් යුත් property එකක් Object එක ඇතුළේ තියෙනවද කියලා. හැබැයි එහෙම එකක් නැති නිසා `undefined` ලැබෙනවා.
- `student[key]` කියද්දී `key` variable එකේ අගය වන `"course"` ආදේශ වී `student["course"]` කියවෙන නිසා නිවැරදිව `"JavaScript"` ලැබෙනවා.

Part D: Adding, Updating, and Deleting Properties

1. Adding and Updating Properties

Concept Explanation (Sinhala): Object එකක් මුලින්ම හැදුවට පස්සේ, අපිට ඕනෑම වෙලාවක අලුත් properties එකතු කරන්න (Add) හෝ තියෙන properties වෙනස් කරන්න (Update) පුළුවන්.

Code Block:

```
const user = {
  name: "Alice",
  score: 10
};

// 1. Adding a new property (අලුත් property එකක් එක් කිරීම):
user.active = true;

// 2. Updating an existing property (තියෙන property එකක් වෙනස් කිරීම):
user.score = 20;

console.log(user);
// Output: { name: "Alice", score: 20, active: true }
```

Code Breakdown (Sinhala):

- `user.active = true;` මඟින් `active` නමින් අලුත් key එකක් සහ එහි value එක `true` ලෙස Object එකට එකතු වෙනවා.
- `user.score = 20;` මඟින් කලින් තිබුණු `score` අගය වෙන `10` වෙනුවට `20` ආදේශ වෙනවා.
- මේ දේම අපිට bracket notation එකෙනුත් කරන්න පුළුවන්: `user["score"] = 20;`.

2. Deleting a Property with `delete`

Concept Explanation (Sinhala): Object එකක තියෙන property එකක් සම්පූර්ණයෙන්ම ඉවත් කරන්න `delete` operator එක පාවිච්චි කරන්න පුළුවන්.

Code Block:

```
const user = {
  name: "Alice",
  password: "secret123"
};

delete user.password;

console.log(user); // Output: { name: "Alice" }
```

Code Breakdown (Sinhala): `delete user.password` මඟින් `user` Object එකෙන් `password` කියන මුළු property එකම (key සහ value දෙකම) ඉවත් කර දමනවා.

ආරක්ෂිත උපදෙසක්: `delete` මඟින් original Object එක කෙළින්ම වෙනස් වෙනවා (mutation). බොහෝ සංවර්ධකයින් (developers) original එක වෙනස් කරන්නේ නැතිව, අදාළ property එක නැති අලුත්ම Object එකක් හදන්න කැමතියි.

3. Creating a New Object Without a Property (Destructuring + Rest)

Concept Explanation (Sinhala): original object එක වෙනස් නොකර, යම් property එකක් ඉවත් කරලා අලුත් object එකක් හදන්න **destructuring** සහ **rest syntax** පාවිච්චි කරන්න පුළුවන්.

Code Block:

```
const user = {
  name: "Alice",
  password: "secret123",
  role: "student"
};

const { password, ...safeUser } = user;

console.log(safeUser); // Output: { name: "Alice", role: "student" }
```

Code Breakdown (Sinhala): `const { password, ...safeUser } = user;` කියන පේළියෙන් වෙන්නේ:

- `user` එකෙන් `password` කියන එක වෙනම ගලවලා ගන්නවා.
- ඉතිරි වෙන හැම property එකක්ම (`name` සහ `role`) එකතු කරලා `safeUser` කියන අලුත්ම Object එකක් හදනවා.
- දැන් original `user` Object එකට කිසිම හානියක් වෙලා නැහැ.

Part E: Checking If a Property Exists

1. Why Simple Truthy/Falsy Checks Can Be Wrong

Concept Explanation (Sinhala): ධුරන්තර වැරදීමක් (common mistake): property එකක් තියෙනවද බලන්න බොහෝ දෙනෙක් `if (user.score)` වගේ truthy/falsy check එකක් ලියනවා. හැබැයි මේක ලොකු අවුලක් වෙන්න පුළුවන්.

Code Block:

```
const user = {
  score: 0
};

if (user.score) {
  console.log("Score exists");
} else {
  console.log("Score missing");
}
// Output: Score missing
```

Code Breakdown (Sinhala): මෙතනදී `score` කියන එක Object එකේ ඇත්තටම තියෙනවා (එහි අගය `0`). හැබැයි JavaScript වල `0` කියන්නේ **falsy value** එකක්. ඒ නිසා `if (user.score)` කියන condition එක false වෙනවා. එවිට score එක තිබුණත් සලකන්නේ නැතිව "Score missing" කියලා print වෙනවා. *(falsy values: false, 0, null, undefined, NaN, "")*

2. Checking With `Object.hasOwn()`

Concept Explanation (Sinhala): යම් Object එකක් සතුව යම් property එකක් සෘජුවම තිබේදැයි බලන්න තියෙන ආරක්ෂිතම සහ නවීනතම ක්‍රමය තමයි `Object.hasOwn()`.

Code Block:

```
const user = {
  score: 0
};

console.log(Object.hasOwn(user, "score")); // Output: true
```

Code Breakdown (Sinhala): `Object.hasOwn(user, "score")` මඟින් `score` හි අගය `0` (falsy) වුණත්, එම property එක Object එකේ සැබවින්ම පවතින නිසා නිවැරදිව `true` ලබා දෙනවා.

3. Other Checks: `hasOwnProperty()` and `in` Operator

Concept Explanation (Sinhala): පැරණි කේතයන්හි `hasOwnProperty()` ක්‍රමය සහ `in` operator එකද දකින්න ලැබෙනවා.

Code Block:

```
const user = {
  name: "Alice",
  score: 0
};

// 1. hasOwnProperty()
console.log(user.hasOwnProperty("name")); // Output: true

// 2. 'in' operator
console.log("score" in user); // Output: true
```

Code Breakdown (Sinhala):

- `hasOwnProperty()` එකත් හොඳයි, හැබැයි සමහර edge cases (උදාහරණයක් ලෙස `Object.create(null)` මඟින් හදන objects වල) මේක වැඩ කරන්නේ නැහැ. ඒ නිසා modern JS වලදී `Object.hasOwn()` වඩාත්ම සුදුසුයි.
- `in` operator එකෙන් වෙන්නේ අදාළ property එක Object එකේ කෙළින්ම තියෙනවද නැත්නම් එහි prototype (inherited properties) හරහා ලැබිලද කියලා දෙකම බලන එකයි.

4. Property Existence Comparison Table

Concept Explanation (Sinhala): කෙටියෙන් මතක තබා ගැනීමට:

ක්‍රමය	සිදුවන දේ	නිර්දේශය
`if (user.score)`	අගය truthy ද බලයි.	අවදානම් සහගතයි (අගය 0, false වැනි දෙයක් නම් වරදී).
`Object.hasOwn(user, "score")`	Object එක සතුව කෙළින්ම property එක තියෙනවද බලයි.	හොඳම සහ ආරක්ෂිතම ක්‍රමය.
`user.hasOwnProperty("score")`	Object එක සතුව කෙළින්ම property එක තියෙනවද බලයි.	භාවිත කළ හැක, නමුත් `Object.hasOwn()` වඩා හොඳයි.
`"score" in user`	Object එකේ හෝ prototype chain එකේ තියෙනවද බලයි.	උරුම වන (inherited) properties ද පරීක්ෂා වේ.

Part F: Nested Objects and Arrays

ඇත්තම දත්ත (real data) ගොඩක් වෙලාවට එකක් ඇතුළේ තව එකක් විදිහට nested (ගොනු වී) පවතී.

1. Nested Objects

Concept Explanation (Sinhala): Object එකක් ඇතුළේ තවත් Object එකක් තිබීම මෙසේ සිදු වේ.

Code Block:

```
const person = {
  name: "Alice",
  contact: {
    email: "alice@example.com",
    phone: {
      work: "098-765-4321"
    }
  }
};

console.log(person.contact.email);      // Output: alice@example.com
console.log(person.contact.phone.work); // Output: 098-765-4321
```

Code Breakdown (Sinhala): `person.contact.phone.work` කියන එක පියවරෙන් පියවර කියවමු:

1. `person` Object එකට යන්න.
2. එහි තියෙන `contact` Object එක ඇතුළට යන්න.
3. එහි තියෙන `phone` Object එක ඇතුළට යන්න.
4. අවසානයේ එහි ඇති `work` property එකේ value එක ලබා ගන්න.

2. Objects With Arrays Inside

Concept Explanation (Sinhala): Object එකක් ඇතුළේ array එකක් තිබීම.

Code Block:

```
const student = {
  name: "Sangeeth",
  courses: ["HTML", "CSS", "JavaScript"]
};

console.log(student.courses[0]); // Output: HTML
console.log(student.courses[2]); // Output: JavaScript
```

Code Breakdown (Sinhala): `student.courses[2]` යනු `student` Object එකෙන් `courses` කියන array එක අරගෙන, එහි index 2 (තුන්වැනි අයිතමය) කියවීමයි.

3. Arrays of Objects

Concept Explanation (Sinhala): Array එකක් ඇතුළේ Objects කිහිපයක් තිබීම. මෙය ප්‍රායෝගිකව ඉතා වැදගත් රටාවකි (pattern).

Code Block:

```
const students = [
  { name: "Ada", score: 95 },
  { name: "Grace", score: 88 },
  { name: "Linus", score: 76 }
];

console.log(students[0].name);  // Output: Ada
console.log(students[1].score); // Output: 88
```

Code Breakdown (Sinhala):

- `students[1]` මඟින් array එකේ index 1 හි ඇති Object එක වෙන `{ name: "Grace", score: 88 }` තෝරාගනී.
- `.score` මඟින් එම තෝරාගත් Object එකේ `score` අගය වෙන `88` ලබාදේ.

Part G: Looping Through Objects

1. `for...in` Loop (Object Keys සඳහා)

Concept Explanation (Sinhala): Object එකක තියෙන keys එකින් එක loop කරලා බලන්න අපි `for...in` loop එක පාවිච්චි කරනවා.

Code Block:

```
const user = {
  name: "Alice",
  active: true,
  score: 95
};

for (const key in user) {
  console.log(key, user[key]);
}
// Output:
// name Alice
// active true
// score 95
```

Code Breakdown (Sinhala):

- Loop එක කැරකෙන හැම වාරයකදීම `key` කියන variable එකට Object එකේ තියෙන key එකක් ලැබෙනවා (පළමුව `"name"`, පසුව `"active"`, අවසානයේ `"score"`).
- අපිට ඒ ඒ key එකේ value එක ගන්න ඕන නිසා අපි **bracket notation** පාවිච්චි කරනවා: `user[key]`. (මෙහිදී `user.key` ලෙස ලියන්න බැහැ, මන්ද Object එකේ කෙළින්ම `key` කියලා property එකක් නැති නිසා).

2. Looping Through an Array of Objects

Concept Explanation (Sinhala): Array එකක් ඇතුළේ Objects තියෙනකොට අපි සාමාන්‍ය `for...of` loop එක පාවිච්චි කරනවා.

Code Block:

```
const students = [
  { name: "Ada", score: 95 },
  { name: "Grace", score: 88 },
  { name: "Linus", score: 76 }
];

for (const student of students) {
  console.log(student.name, student.score);
}
// Output:
// Ada 95
// Grace 88
// Linus 76
```

Code Breakdown (Sinhala):

- `for...of` මඟින් `students` array එකේ තියෙන හැම student Object එකක්ම එකින් එක `student` variable එකට ලබාදෙනවා.
- ඉන්පසු `student.name` සහ `student.score` ලෙස ඒවායේ තොරතුරු මුද්‍රණය කරයි.

මතක තබාගත යුතු රීතිය:

- Object එකක keys හරහා යන්න: `for...in` පාවිච්චි කරන්න.
- Array එකක අයිතම හරහා යන්න: `for...of` පාවිච්චි කරන්න.

Part H: Primitive vs Non-Primitive Values

JavaScript වල දත්ත වර්ග දෙකකි: **Primitive** සහ **Non-Primitive**.

1. Primitive Values

Concept Explanation (Sinhala): Primitive දත්ත (strings, numbers, booleans, null, undefined) variable එකකින් තව එකකට assign කරනකොට, එහි අගය සැබවින්ම පිටපත් (copied by value) වෙනවා.

Code Block:

```
let a = 10;
let b = a; // 'a' හි ඇති අගය (10) 'b' වෙත අලුතින්ම copy වේ.
a = 20;

console.log(a); // Output: 20
console.log(b); // Output: 10
```

Code Breakdown (Sinhala): `a` හි අගය වෙනස් කළත් `b` හි අගය වෙනස් නොවේ. මන්ද `b` සතුව ඇත්තේ ස්වාධීන වෙනම 10 අගයක් ඇති copy එකක් නිසාය.

2. Non-Primitive Values (Objects Are Copied by Reference)

Concept Explanation (Sinhala): Objects (සහ arrays, functions) යනු Non-primitive දත්ත වේ. මේවා assign කිරීමේදී පිටපත් වන්නේ අගය නොවේ, මතකයේ එය පවතින ස්ථානය පෙන්වන ලිපිනයයි **(copied by reference)**.

Code Block:

```
const user = { name: "Alice" };
const sameUser = user; // මෙතනදී අලුත් Object එකක් හැදෙන්නේ නැත. ලිපිනය (reference) පමණක් copy වේ.

user.name = "Ada";

console.log(sameUser.name); // Output: Ada
```

Code Breakdown (Sinhala): `user` සහ `sameUser` කියන variables දෙකම පෙන්වන්නේ මතකයේ තියෙන එකම එක Object එකකටයි. ඒ නිසා `user.name` වෙනස් කළ සැනින් `sameUser.name` ද වෙනස් වේ.

3. `const` with Objects

Concept Explanation (Sinhala): `const` මඟින් Object එකක් හැදුවත්, ඒ Object එක ඇතුළේ තියෙන properties වෙනස් කරන්න අපිට පුළුවන්. `const` මඟින් වළක්වන්නේ එම variable එකට වෙනත් අලුත්ම Object එකක් නැවත assign කිරීම (reassignment) පමණයි.

Code Block:

```
const user = { name: "Alice" };

user.name = "Ada"; // මේක කරන්න පුළුවන්! (property modification)

// user = { name: "Grace" }; // මේක කරන්න බැහැ! TypeError එකක් එනවා (reassignment).
```

Part I: Methods

1. What Is a Method?

Concept Explanation (Sinhala): Method එකක් කියන්නේ Object එකක් ඇතුළේ store කරලා තියෙන function එකකටයි.

Code Block:

```
const calculator = {
  // Long version:
  add: function(a, b) {
    return a + b;
  },
  // Short version (නවීන කෙටි ක්‍රමය):
  subtract(a, b) {
    return a - b;
  }
};

console.log(calculator.add(2, 3));      // Output: 5
console.log(calculator.subtract(10, 5)); // Output: 5
```

2. `this` Inside a Method

Concept Explanation (Sinhala): Method එකක් ඇතුළේ `this` keyword එක පාවිච්චි කළ විට, එයින් අදහස් කරන්නේ එම method එක අයිති ප්‍රධාන Object එකයි.

Code Block:

```
const student = {
  name: "Sangeeth",
  scores: [90, 85, 100],
  getAverageScore() {
    let total = 0;
    // 'this.scores' මඟින් student Object එකේ 'scores' array එක ලබාදේ.
    for (const score of this.scores) {
      total += score;
    }
    return total / this.scores.length;
  }
};

console.log(student.getAverageScore()); // Output: 91.66666666666667
```

Code Breakdown (Sinhala): `this.scores` කියන්නේ `student.scores` කියන එකටමයි. Method එකක් ඇතුළත සිට එම Object එකේම වෙනත් properties access කිරීමට `this` අත්‍යවශ්‍ය වේ.

Part J: The Object() Constructor

Concept Explanation (Sinhala): අලුතින් Object එකක් හදන්න `new Object()` කියන constructor එක පාවිච්චි කරන්නත් පුළුවන්. හැබැයි beginner-friendly සහ හොඳම ක්‍රමය තමයි සාමාන්‍ය සඟල වරහන් පාවිච්චි කරන **object literal syntax** එක.

Code Block:

```
// Object Literal Syntax (හොඳම සහ සරලම ක්‍රමය):
const user1 = { name: "Alice" };

// Object Constructor Syntax (අනවශ්‍ය ලෙස දිගු ක්‍රමය):
const user2 = new Object();
user2.name = "Alice";
```

වැදගත්: `Object(null)` හෝ `Object(undefined)` ලෙස දුන්නොත් ලැබෙන්නේ හිස් Object `{}` එකකි. නමුත් ප්‍රායෝගික කේතකරණයේදී මෙවැනි දේ භාවිත කිරීමෙන් වළකින්න.

Part K: JSON (JavaScript Object Notation)

1. What Is JSON?

Concept Explanation (Sinhala): JSON කියන්නේ දත්ත ගබඩා කිරීමට හෝ server එකක් සමඟ හුවමාරු කිරීමට භාවිත කරන සරල **text format** (ලිඛිත ආකෘතියක්) එකක්. එය බැලූ බැල්මට JavaScript Object එකක් වගේ පෙනුනත්, එය සැබවින්ම string (text) එකක් පමණි.

2. JSON Syntax Rules (දැඩි නීති රීති)

Concept Explanation (Sinhala):

1. හැම key (property name) එකකටම අනිවාර්යයෙන්ම **double quotes** `""` තිබිය යුතුය.
2. String values සඳහාද අනිවාර්යයෙන්ම **double quotes** `""` තිබිය යුතුය. (single quotes `''` හෝ backticks ` `` ` බැහැ).
3. JSON ඇතුළේ functions store කරන්න බැහැ.
4. JSON ඇතුළේ comments ලියන්න බැහැ.
5. අන්තිම property එකට පස්සේ trailing comma (අන්තිම කොමාව `,`) තියන්න බැහැ.

Code Block:

```
// නිවැරදි JSON ආකෘතිය:
{
  "name": "Alice",
  "active": true,
  "score": 95,
  "courses": ["HTML", "CSS"]
}
```

3. Converting JSON: `JSON.stringify()` and `JSON.parse()`

Concept Explanation (Sinhala):

- `JSON.stringify(object)` : JavaScript Object එකක් JSON text එකක් බවට හරවයි (Server එකකට යැවීමට හෝ save කිරීමට).
- `JSON.parse(jsonText)` : JSON text එකක් නැවත JavaScript Object එකක් බවට හරවයි (භාවිතයට ගැනීමට).

Code Block:

```
const user = {
  name: "Alice",
  active: true
};

// 1. Object to JSON string:
const jsonString = JSON.stringify(user);
console.log(jsonString); // Output: '{"name":"Alice","active":true}'
console.log(typeof jsonString); // Output: string

// 2. JSON string back to Object:
const parsedObject = JSON.parse(jsonString);
console.log(parsedObject.name); // Output: Alice
```

4. JSON formatting with Spacing (Pretty Print)

Concept Explanation (Sinhala): `JSON.stringify()` කරනකොට කියවන්න ලේසි වෙන්න spacing (indentation) එකතු කරන්න පුළුවන්.

Code Block:

```
const user = { name: "Alice", active: true };
const prettyJson = JSON.stringify(user, null, 2);
console.log(prettyJson);
/* Output:
{
  "name": "Alice",
  "active": true
}
*/
```

5. JSON with Replacer (දත්ත පෙරීම)

Concept Explanation (Sinhala): අපිට Object එකකින් JSON string එක හදනකොට, ඇතුළත් වෙන්න ඕන නිශ්චිත properties විතරක් තෝරලා දෙන්න පුළුවන්.

Code Block:

```
const user = {
  name: "Alice",
  password: "secret123",
  role: "student"
};

const safeJson = JSON.stringify(user, ["name", "role"], 2);
console.log(safeJson);
/* Output:
{
  "name": "Alice",
  "role": "student"
}
*/
```

Part L: Optional Chaining (`?.`)

1. The Problem Optional Chaining Solves

Concept Explanation (Sinhala): සමහර අවස්ථාවලදී අපේ Object එකක යම් nested property එකක් නැති වෙන්න පුළුවන් (උදාහරණයක් විදිහට API එකකින් එන දත්ත අසම්පූර්ණ වීම). එතකොට කෙළින්ම access කරන්න ගියොත් මුළු app එකම crash වෙනවා.

Code Block:

```
const user = {
  name: "Alice"
  // profile property එකක් මෙතන නැහැ!
};

// console.log(user.profile.email); // මෙයෙන් මුළු app එකම crash වෙනවා! (TypeError)
```

Code Breakdown (Sinhala): `user.profile` කියන එක `undefined` වෙනවා. ඉන්පසු `undefined.email` කියවා ගැනීමට උත්සාහ කිරීම නිසා JavaScript error එකක් දෙනවා.

2. Optional Chaining `?.` Syntax

Concept Explanation (Sinhala): Optional Chaining (`?.`) මඟින් යම් තැනක දත්ත නැතිනම් (null හෝ undefined නම්) එතැනින් ඉදිරියට කියවීම නවත්වා, crash නොවී සරලව `undefined` ලබා දෙයි.

Code Block:

```
const user = {
  name: "Alice"
};

console.log(user.profile?.email); // Output: undefined (crash වෙන්නේ නැහැ!)
```

Code Breakdown (Sinhala): `user.profile?.email` කියන එකෙන් අහන්නේ: "user එකේ profile එකක් තියෙනවද? තියෙනවා නම් එහි email එක දෙන්න. නැත්නම් එතනින්ම නතර වෙලා `undefined` දෙන්න."

3. Optional Chaining with Arrays

Concept Explanation (Sinhala): Array එකක් ඇතුළේ නියෙන index එකක් access කිරීමටද මෙය යොදාගත හැක.

Code Block:

```
const student = {
  name: "Sangeeth",
  courses: [{ title: "HTML" }]
};

console.log(student.courses?.[1]?.title); // Output: undefined (crash නොවේ)
```

මතක තබාගන්න: Optional chaining මඟින් කරන්නේ crash වීම් වැළැක්වීම පමණි. එයින් අලුතින් දත්ත නිර්මාණය කරන්නේ නැත.

Part M: Object Destructuring

1. What Is Object Destructuring?

Concept Explanation (Sinhala): Object destructuring මඟින් අපිට Object එකක තියෙන properties ඉතා පහසුවෙන් වෙනම variables වලට ගලවා ගන්න (extract කරන්න) පුළුවන්.

Code Block:

```
const user = {
  name: "Alice",
  age: 25
};

// Destructuring:
const { name, age } = user;

console.log(name); // Output: Alice
console.log(age);  // Output: 25
```

Code Breakdown (Sinhala): `const { name, age } = user;` මඟින් `user` Object එකේ ඇති `name` සහ `age` properties වල නම් වලින්ම අලුත් variables දෙකක් සාදා ඒවාට අදාළ අගයන් ලබා දෙයි. (මෙහිදී array destructuring වගේ index පිළිවෙළ වැදගත් නැත. වැදගත් වන්නේ property name එක ගැලපීම පමණි).

2. Destructuring with Default Values and Renaming

Concept Explanation (Sinhala): Destructuring කරන ගමන්ම අපිට variable එකට වෙනත් නමක් දෙන්නත් පුළුවන් (Renaming), එමෙන්ම අදාළ property එක නැතිනම් ලැබිය යුතු default value එකක් දෙන්නත් පුළුවන්.

Code Block:

```
const user = {
  name: "Alice"
};

const { name: userName, country = "Unknown" } = user;

console.log(userName); // Output: Alice
console.log(country);  // Output: Unknown
// console.log(name);  // ReferenceError! (නම userName ලෙස වෙනස් කළ නිසා 'name' නමින් variable එකක් නැත)
```

Code Breakdown (Sinhala):

- `name: userName` මඟින් `name` property එකේ අගය අරගෙන `userName` නමින් variable එකක් හදනවා.
- `country = "Unknown"` මඟින් Object එකේ `country` නැති නිසා, එයට `"Unknown"` කියන default value එක ලබා දෙනවා.

3. Nested Destructuring (අවදානම සහ විසඳුම)

Concept Explanation (Sinhala): Nested object එකක් destructure කිරීමේදී, මව් object එක (parent object) නොමැති නම් code එක crash විය හැක.

Code Block:

```
const user = {
  name: "Alice"
  // contact නැහැ!
};

// Risky Nested Destructuring (අවදානම් සහිතයි):
// const { contact: { email } } = user; // TypeError එකක් එනවා!

// Safer beginner version (ආරක්ෂිත ක්‍රමය):
const email = user.contact?.email;
console.log(email); // Output: undefined
```

Part N: Object Property Shorthand

Concept Explanation (Sinhala): අපි Object එකක් හදනකොට, key එකේ නම සහ අපි ආදේශ කරන්න හදන variable එකේ නම සමාන නම්, අපිට ඒක දෙපාරක් ලියන්න ඕනේ නැහැ **(Property Shorthand)**.

Code Block:

```
const name = "Alice";
const age = 25;

// Long version:
const userLong = {
  name: name,
  age: age
};

// Shorthand version (කෙටි ක්‍රමය):
const userShort = {
  name,
  age
};

console.log(userShort); // Output: { name: "Alice", age: 25 }
```

Part Q: Common Beginner Mistakes (ආධුනිකයින් කරන ප්‍රධාන වැරදි)

1. Mistake: Dynamic key සඳහා Dot Notation පාවිච්චි කිරීම

- වැරදි ක්‍රමය: `user.key` (මෙහිදී `key` කියන අකුරු වලින්ම ඇති property එකක් හොයයි).
- නිවැරදි ක්‍රමය: `user[key]` (මෙහිදී `key` variable එක ඇතුළේ ඇති value එක අදාළ කරගනී).

2. Mistake: `const` භාවිතයෙන් Object immutable (වෙනස් කළ නොහැකි) වේ යැයි සිතීම

- `const` මඟින් වළක්වන්නේ variable එක වෙනස් කිරීම පමණි. Object එක ඇතුළේ ඇති properties වෙනස් කිරීමට පුළුවන.

3. Mistake: Falsy logic මඟින් property පැවැත්ම පරීක්ෂා කිරීම

- `if (user.score)` ලෙස ලියූ විට score අගය `0` වුවහොත් එය false ලෙස සලකයි.
- නිවැරදි ක්‍රමය: `if (Object.hasOwn(user, "score"))`.

4. Mistake: JSON තුළ functions ගබඩා කළ හැකැයි සිතීම

- JSON මඟින් ගබඩා කරන්නේ දත්ත (data) පමණි, ක්‍රියාකාරකම් (behavior / functions) ගබඩා කළ නොහැක.

Part S: Mini-Builds

Mini-Build 1: Student Report (සාමාන්‍ය අගය සෙවීම)

Code Block:

```
const student = {
  name: "Sangeeth",
  scores: [90, 85, 100],
  getAverage() {
    let total = 0;
    for (const score of this.scores) {
      total += score;
    }
    return total / this.scores.length;
  }
};

console.log(`${student.name}'s average is ${student.getAverage()}`);
// Output: Sangeeth's average is 91.66666666666667
```

Sinhala Explanation: `this.scores` මඟින් Object එක ඇතුළේ ඇති array එක ලබාගනී. ඉන්පසු loop එකක් මඟින් සියල්ල එකතු කර සාමාන්‍ය අගය (average) ගණනය කර ආපසු ලබාදෙයි.

Mini-Build 2: Safe User Summary (මුද්‍රණයේදී රහස්‍ය දත්ත සැඟවීම)

Code Block:

```
const user = {
  name: "Alice",
  email: "alice@example.com",
  password: "secret123",
  role: "student"
};

// password ඉවත් කර safeUser සාදා ගනිමු:
const { password, ...safeUser } = user;
console.log(safeUser);
// Output: { name: "Alice", email: "alice@example.com", role: "student" }
```

Sinhala Explanation: මෙතනදී rest operator (`...safeUser`) මඟින් `password` හැර අනෙකුත් සියලුම properties අඩංගු නව Object එකක් සාදයි.

Part V: Minimum Passing Standard (අවම සමත් වීමේ මට්ටම)

ඔයාට Sprint 8 එක සාර්ථකව අවසන් කරන්න නම් පහත දැක්වෙන කේතයේ (code) සෑම පේළියක්ම පැහැදිලි කිරීමට හැකි විය යුතුය.

Code Block:

```
const user = {
  name: "Alice",
  score: 0,
  contact: {
    email: "alice@example.com"
  },
  courses: [
    { title: "HTML", complete: true },
    { title: "JavaScript", complete: false }
  ],
  getSummary() {
    return `${this.name} has ${this.courses.length} courses.`;
  }
};

console.log(user.name);                       // Output: Alice
console.log(user.contact?.email);             // Output:
alice@example.com
console.log(user.profile?.city);              // Output: undefined
console.log(user.courses[1].title);           // Output: JavaScript
console.log(Object.hasOwn(user, "score"));    // Output: true
console.log(user.getSummary());               // Output: Alice has 2
courses.

const { name, country = "Unknown" } = user;
console.log(name);                            // Output: Alice
console.log(country);                         // Output: Unknown

const json = JSON.stringify(user, null, 2);
const parsed = JSON.parse(json);
console.log(parsed.name);                     // Output: Alice
```

Code Breakdown (Sinhala):

1. `user.name` -> Dot notation මඟින් සෘජුවම name අගය ගනී.
2. `user.contact?.email` -> `contact` පවතින නිසා එහි ඇති `email` ලබාදේ.
3. `user.profile?.city` -> `profile` නොමැති වුවත්, optional chaining නිසා crash නොවී `undefined` දෙයි.
4. `user.courses[1].title` -> `courses` array එකේ index 1 හි ඇති Object එකේ `title` අගය ("JavaScript") ලබාදේ.
5. `Object.hasOwn(user, "score")` -> `score` හි අගය `0` වුවත්, එම property එක පවතින නිසා `true` ලබාදේ.
6. `user.getSummary()` -> Method එක ක්‍රියාත්මක වී summary string එක ලබාදේ.
7. `const { name, country = "Unknown" } = user` -> Destructuring මඟින් `name` ලබාගන්නා අතර, `country` නොමැති නිසා default value එක ආදේශ කරගනී.
8. `JSON.stringify(user, null, 2)` -> මුළු Object එකම ලස්සනට format කළ JSON text එකක් බවට පත් කරයි (මෙහිදී `getSummary` function එක ඇතුළත් නොවේ).
9. `JSON.parse(json)` -> එම JSON text එක නැවත සාමාන්‍ය JS Object එකක් බවට හරවයි.